



System design

Dropdown Menu ★ Premium

Medium 30 mins 810 done

Report an issue

Asked at these companies



Google



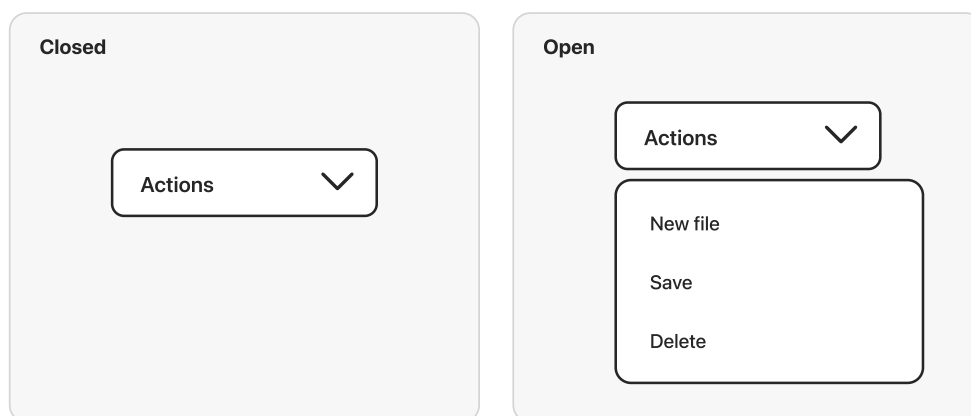
ByteDance



TikTok

Question

Design a dropdown menu component that can reveal a menu containing a list of actions.



Real-life examples

These component libraries demonstrate common dropdown-menu APIs and interaction patterns.

- [Dropdowns · Bootstrap v5.3](#)
- [React Menu component - Material UI](#)

- Menu — Base UI
-

Requirements exploration

Before designing the API, scope the component by clarifying how many menus can coexist, what content they hold, which devices they support, and the expected level of customization.

Can there be multiple dropdown menus open at once?

Yes, there can be.

What content will the menu contain?

Only text and icons, with no nested interactive controls. The menu items themselves still participate in managed focus.

Is there a maximum number of items allowed in the menu?

There's no fixed maximum, but preferably, there should not be more than 20 items for a better user experience.

What amount of flexibility does the user have in customizing the design?

Users should be able to customize colors, fonts, padding, etc., to match their branding.

What devices will this component be used on?

All devices: mobile, tablet, desktop.

Exercise Beta

A team is defining the first version of a reusable dropdown menu. Which requirements belong in it? Select all that apply.

- 1 Allow more than one dropdown menu to remain open at the same time. ☐
- 2 Treat text fields, checkboxes, and other nested controls as standard menu-item children in the first version. ☐
- 3 Treat roughly twenty items as a usability preference rather than a hard component limit. ☐
- 4 Cap each menu at twenty items and require consumers to move remaining actions into a second menu. ☐
- 5 Support text and icons as item content, without nested interactive controls inside items. ☐
- 6 Support mobile, tablet, and desktop use. ☐



Reveal answer

Check answer ↩

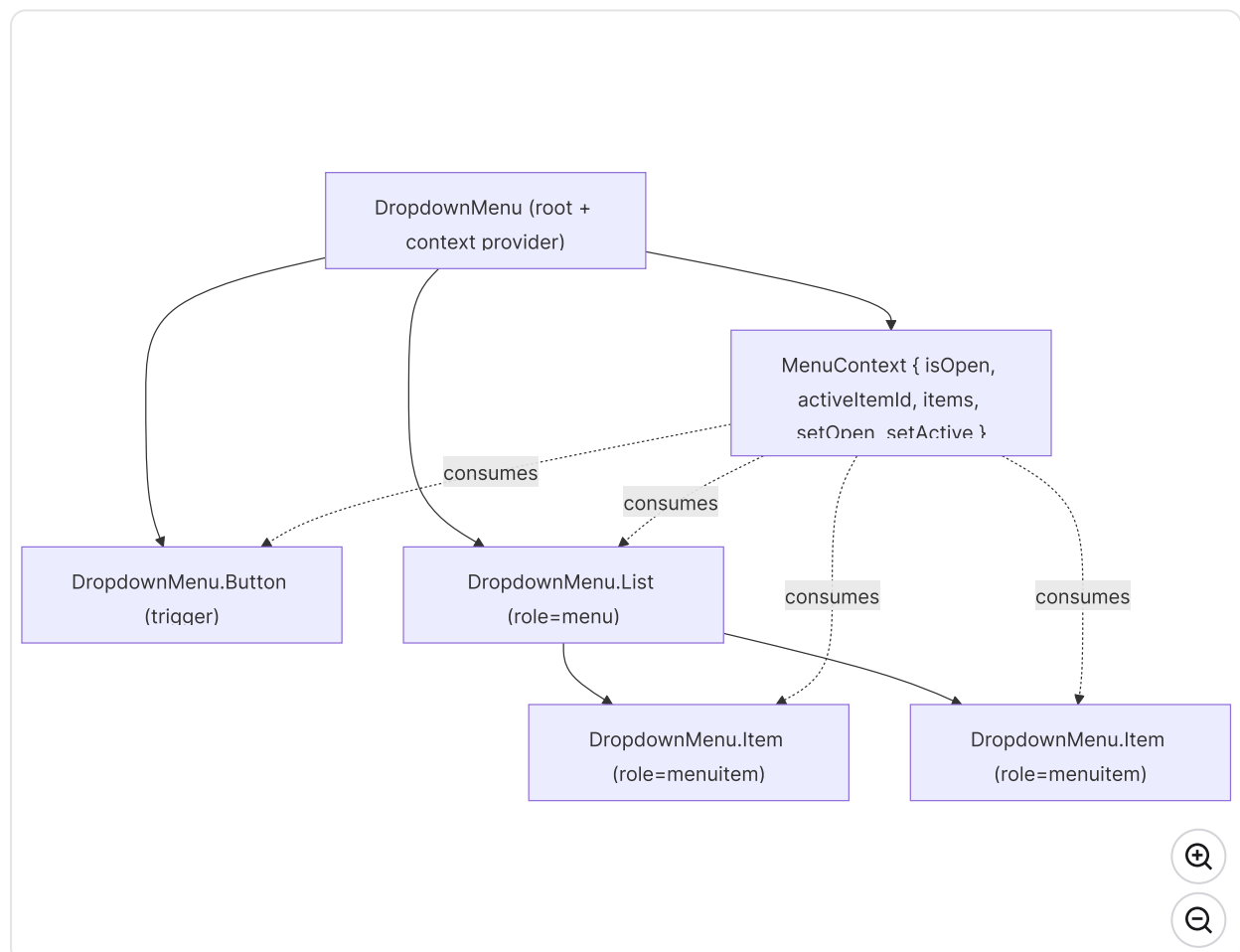
Architecture / high-level design

An example usage of the dropdown menu in React, with event handlers omitted.

```
1 <DropdownMenu>
2   <DropdownMenu.Button>Actions</DropdownMenu.Button>
3   <DropdownMenu.List>
4     <DropdownMenu.Item>New File</DropdownMenu.Item>
5     <DropdownMenu.Item>Save</DropdownMenu.Item>
6     <DropdownMenu.Item>Delete</DropdownMenu.Item>
7   </DropdownMenu.List>
8 </DropdownMenu>
```

Component	Role
Dropdown Menu (<code>DropdownMenu</code>)	Root component, coordinates events between the inner components.
Menu Button (<code>DropdownMenu.Button</code>)	Button that toggles the display state of the <code>DropdownMenu.List</code> .
Menu List (<code>DropdownMenu.List</code>)	Contains the list of items.
Menu List Item (<code>DropdownMenu.Item</code>)	Individual list items.

In React, components can communicate with their parents using context or props. Context fits this composition model because it avoids threading shared menu state through every subcomponent. `<DropdownMenu>` should contain a context provider that supplies the state values from the Data model section to its descendants.



- ★ **Expose a compound-component API rather than a single monolithic prop tree**
- Splitting the component into `DropdownMenu`, `DropdownMenu.Button`, `DropdownMenu.List`, and `DropdownMenu.Item` lets consumers compose arbitrary trigger and item content while the root owns shared state via context. A single `<DropdownMenu items={...} />` prop-based API quickly breaks down once items need icons, disabled states, or custom rendering.

Exercise Beta

A reusable dropdown renders its trigger, floating list, and items as separate parts. Which responsibility split keeps their behavior coherent?

- 1 Let one root own open state, item registration, and focus transitions; each part only reports semantic events and renders supplied state.
- 2 Query the rendered trigger and popup for every event, then infer open state, item order, and the next focus target from those elements.
- 3 Let the trigger own open state, each item handle keyboard movement, and the list own dismissal, with every part publishing state changes independently.

[Reveal answer](#)[Check answer ↩](#)

Data model

Note that when designing components, it might make sense to design the interface/API first, or both the data model and API concurrently. It depends on the component at hand. Feel free to jump between the two sections.

State	Type	Description
<code>isOpen</code>	<code>boolean</code>	Whether the menu is currently open or closed.

State	Type	Description
<code>activeItemId</code>	<code>string null</code>	Stable ID of the managed-focus item, or <code>null</code> when no item is active.
<code>items</code>	ordered registry	Registered item IDs, DOM refs, disabled state, and text values used for focus movement and typeahead.
<code>triggerId</code> / <code>listId</code>	<code>string</code>	Stable IDs connecting the trigger and menu.
<code>triggerRef</code> / <code>listRef</code>	DOM refs	References used for focus restoration and portaled interaction boundaries.

These state values are housed within `<DropdownMenu>` and provided to all components via React context.

Each item registers `{ id, ref, disabled, textValue }` on mount and unregisters on removal; the root maintains DOM order and derives navigation and typeahead from that registry. If the active item disappears, select the nearest remaining item or clear the active ID. This design uses programmatic item focus: every item renders `role="menuitem"` with `tabindex="-1"`, and the root calls `ref.current?.focus()` for the active item rather than using `aria-activedescendant`.

See below for configuration options, which are also part of the data model.

Exercise 1 of 2 Beta

A menu supports typeahead, disabled items, and a portaled list. Which values belong in its interaction model? Select all that apply.

- 1

Ordered item metadata including disabled state and text used for typeahead.

☐
- 2

The active item identity for keyboard movement and programmatic item focus.

☐

3

Serialize the current popup DOM whenever it opens and use that snapshot for keyboard order until dismissal.

☐

4

Whether the popup is open and which trigger owns it.

☐

5

Use the latest pointer coordinates as active-item state and derive the item by hit testing when selection occurs.

☐

Reveal answer

Check answer ↩

Interface definition (API)

The API surface is split into props shared across subcomponents and props specific to each composition piece (root, trigger, menu, item).

General props

These props apply to most of the components.

Prop	Type	Description
<code>children</code>	<code>React.Node</code>	Children of the component. If using TypeScript/Flow, you can enforce specific components to be used as <code>children</code> .
<code>as</code>	<code>string Component</code>	In case there is a need to customize the underlying DOM element/component.
<code>className</code>	<code>string</code>	Classnames to add to components in case further visual customization is needed. May or may not be needed depending on theming approach.

DropdownMenu

The root supports both controlled and uncontrolled open state.

Prop	Type	Description
<code>open</code> / <code>defaultOpen</code>	<code>boolean</code>	Controlled or initial uncontrolled open state.
<code>onOpenChange</code>	<code>function</code>	Called whenever the root requests an open-state change.
<code>size</code>	<code>string</code>	Prop to customize size. Needed only if customization is desired.

DropdownMenu.Button

The trigger forwards its DOM ref and ID so the root can connect ARIA state and restore focus when appropriate.

Prop	Type	Description
<code>onClick</code>	<code>function</code>	Although the opening/closing will be handled within <code>DropdownMenu</code> , exposing an <code>onClick</code> prop is useful if additional logic needs to be performed (e.g., analytics logging).
Other <code>button</code> props	*	Since this component is usually a <code><button></code> , it should also allow other props that <code><button></code> s expect, such as <code>disabled</code> .

DropdownMenu.List

The list exposes placement and size options while the root owns its generated ID and portaled ref.

Prop	Type	Description
<code>maxHeight</code>	<code>number</code> <code>undefined</code>	Max height of the menu list. There should be a sensible default of around 200-300px.
<code>position</code>	<code>string</code>	Position of the list relative to the button.

DropdownMenu.Item

Each item registers its focus and typeahead metadata and exposes one activation callback for pointer and keyboard selection.

Prop	Type	Description
<code>onSelect</code>	<code>function</code>	Triggered for pointer or keyboard activation. The action determines whether focus returns to the trigger or moves to a resulting context.
<code>disabled</code>	<code>boolean</code>	Whether the item is disabled. Disabled items remain reachable by arrow-key navigation but cannot be activated.
<code>textValue</code>	<code>string</code>	Text used by the root's typeahead behavior when rendered content is not plain text.

Customizing appearance

Guidance on designing good APIs for customizing UI components can be found in the [Front End Interview Guidebook's UI Components API Design Principles Section](#).

Exercise 1 of 3 Beta

A reusable dropdown lets consumers customize the trigger, list, and item presentation. Which public contract preserves one interaction model across those parts?

1

Accept arbitrary visual content but discover actions and disabled state only by parsing its rendered HTML.

2

Expose global open and close functions that locate a menu by DOM ID while items register locally.

3

Expose explicit root, trigger, list, and item contracts, controlled open and selection callbacks, and styling hooks for each visual part.



Reveal answer

Check answer ↩

Optimizations and deep dive

The tricky parts of a dropdown menu are floating rendering, viewport-aware positioning, accessibility, and language flexibility:

- **Rendering**: This section explains how to render the floating menu outside normal document flow while preserving positioning and layering behavior.
- **Automatic flipping when near the edge**: This section covers viewport collision detection and placement changes when there is not enough space.
- **Accessibility (a11y)**: This section discusses pointer, keyboard, and assistive-technology behavior for a usable dropdown menu.
- **Internationalization (i18n)**: This section covers long translated labels, text direction, and layout flexibility for localized menus.

The embedded sandboxes illustrate layout and dismissal only; use the surrounding prose as the accessibility contract. The frames request no device or payment permissions.

Rendering

Rendering the dropdown menu can be quite tricky because the menu is "floating" and does not exactly follow the normal flow of page elements.

Layout

There are two common ways to render the dropdown menu near the button. The samples below illustrate their layout differences.

Relative to Button

In this approach, a `<div>` with `position: relative` wraps the `<button>` and menu. The menu uses `position: absolute`, which positions it relative to that root `<div>`.

This method is straightforward and does not require much calculation of elements and their positions relative to the page, but parent containers with `overflow: hidden` can clip the menu and its contents, or there could be `z-index` issues.

Relative to Page

In this approach, the menu is rendered as a direct child of the `<body>`. `offsetTop` and `offsetLeft` are relative to an `offsetParent`, not necessarily the page, so a portaled menu should use one explicit coordinate system: viewport coordinates from

`getBoundingClientRect()` with fixed positioning, or document coordinates after adding the page scroll offsets.

In React, this can be done using **React Portals**, which let you render outside the DOM hierarchy of the parent component. A typical use case for portals is when a parent component has an `overflow: hidden` or `z-index` style, but you need the child to visually "break out" of its container. Common examples include dropdown menus, tooltips, and modals.

The downside is that the position must be updated while the menu is open. A maintained positioning primitive can own main-axis flipping, cross-axis shifting, available-size constraints, updates after ancestor scroll/resize, element resize, or layout shift, and cleanup on close. Exact observer wiring is a production follow-up.

✓ **Prefer portal-based rendering for production-grade dropdown menus**

Rendering the menu as a child of `<body>` via a portal avoids clipping from ancestor `overflow: hidden` containers and sidesteps `z-index` stacking-context bugs that plague the relative-to-button approach. The position-recalculation cost on resize or

scroll is a much easier problem to solve than chasing container-specific clipping across every page the component ships on.

Position

The component can allow customization of alignment in all directions around the `<button>`. The relative-position example above shows left- and right-aligned lists. Model those choices as a preferred side plus logical `start / end` alignment so the collision policy can still flip or shift the placement when it would overflow.

Maximum height

Since there is no maximum allowable number of items in the menu, set a default maximum height so excess items remain accessible through scrolling. This height can also be customized.

Rendering in HTML or JavaScript

The dropdown menu can either be:

1. Rendered into the HTML like **Bootstrap's Dropdowns**. The menu is initially hidden with `display: none` or the `hidden` attribute. `opacity: 0` alone is insufficient because the menu can remain interactive and exposed to assistive technology.
2. Rendered on the fly via JavaScript after the menu button is activated.

The pro of rendering in the HTML first is better runtime performance due to fewer DOM operations needed to show the menu. The downside is that the HTML can be unnecessarily bloated, especially if the user never interacts with the dropdown menu at all. Since the menu items usually don't contribute towards SEO and there likely won't be that many elements, the benefits of rendering the menu beforehand in HTML are relatively minor.

Exercise 1 of 2 Beta

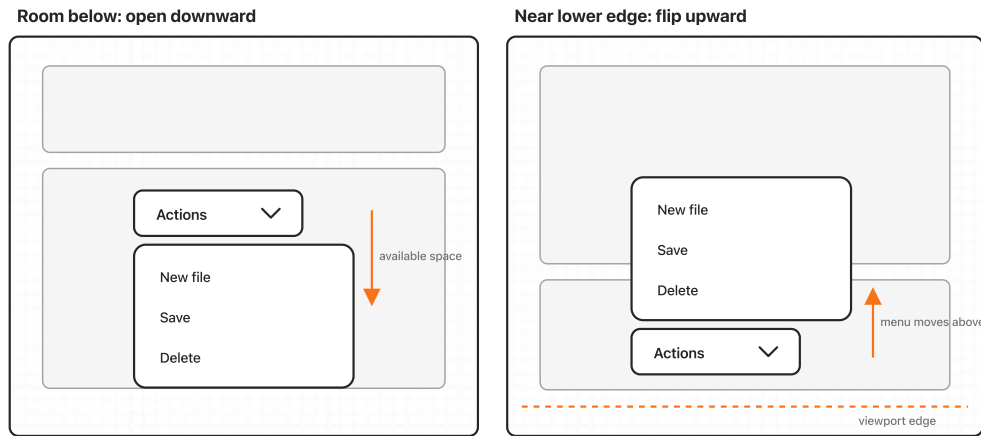
A trigger sits inside an `overflow: hidden` panel that can scroll and resize. Which choices keep its popup visible and attached? Select all that apply.

- ☐ 1 Render the popup in a portal or dedicated overlay layer when ancestors can clip it.
- ☐ 2 Keep the popup beside the trigger and create a new local stacking context with a very large z-index.
- ☐ 3 Measure trigger and popup geometry and update placement on relevant resize or scroll changes.
- ☐ 4 Measure when the popup opens and keep that placement until it closes, even if an ancestor scrolls.

[Reveal answer](#)[Check answer ↩](#)

Automatic flipping when near the edge

Smart dropdown menus know their position relative to the viewport and can flip or shift when there is insufficient space. Disabling page scrolling is not a substitute for collision handling.



The basic vertical check combines the viewport height with the trigger's position and height, the menu height, and the spacing between them:

- **Viewport height:** `window.innerHeight` .
- **Position of the menu's bottom edge:**
 1. Trigger position relative to the viewport
(`buttonEl.getBoundingClientRect().y`)
 2. Trigger height (`buttonEl.getBoundingClientRect().height`)
 3. Menu height (`menuEl.getBoundingClientRect().height`)
 4. Spacing between trigger and menu

That check demonstrates vertical flipping only. The complete positioner must measure the trigger and menu in the same coordinate space, shift or clamp on the cross axis, and constrain the available width or height when neither side fully fits.

Here's a simplified React example of the vertical autoflip behavior. It is a geometry demonstration, not the complete production positioning contract above.

Exercise 1 of 2 Beta

A dropdown would overflow below the viewport but has space above. What should the positioner do?

- 1 Keep the preferred bottom placement and add an internal scrollbar for whatever portion remains visible.
- 2 Move the trigger upward with a transform so the preferred placement still fits.
- 3 Flip above the trigger and clamp the popup within the available viewport padding.

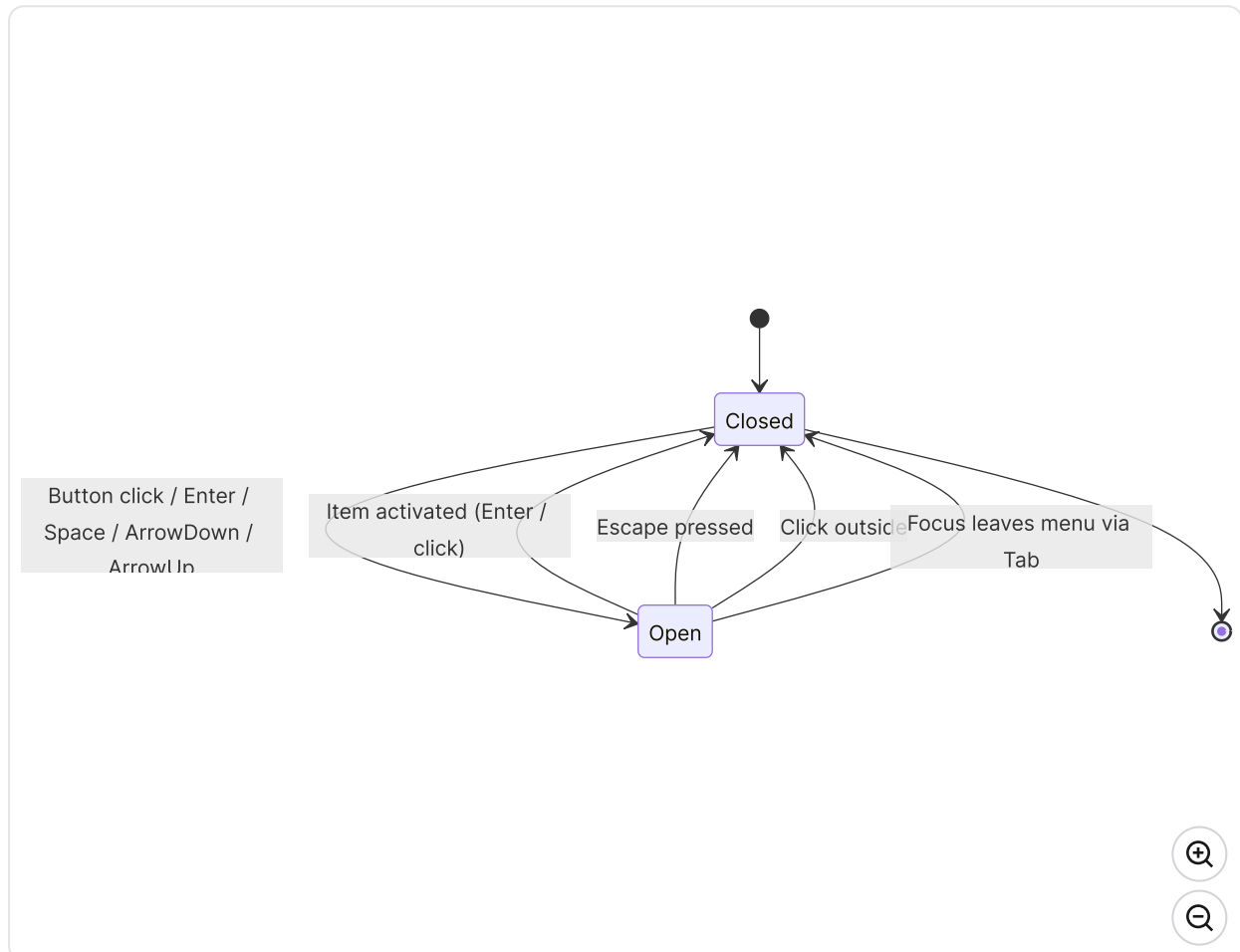
[Reveal answer](#)[Check answer ↩](#)

Accessibility (a11y)

Dropdown menus must be usable with mouse, keyboard, and assistive tech. The subsections below cover each input modality and the ARIA semantics that tie them together.

Mouse interactions

Clicking the button toggles the menu. A completed click outside the registered dropdown group closes open menus, while clicks within any registered trigger or menu stay inside the group.



Menu open and close lifecycle

When multiple menus are allowed to remain open, a document-level coordinator registers every trigger and portaled menu branch, including triggers whose menus are currently closed. Therefore, clicking a sibling trigger is not an outside interaction and can open another menu without closing existing siblings. Using the completed click also avoids dismissing during a canceled drag or scroll gesture.

```
1 function clickListener(event) {
2   const path = event.composedPath();
3   if (registeredMenuBranches.some((element) => path.includes(element)))
```

```
4     return;  
5   }  
6  
7   closeOpenMenusForOutsideInteraction();  
8 }  
9  
10 document.addEventListener("click", clickListener);
```

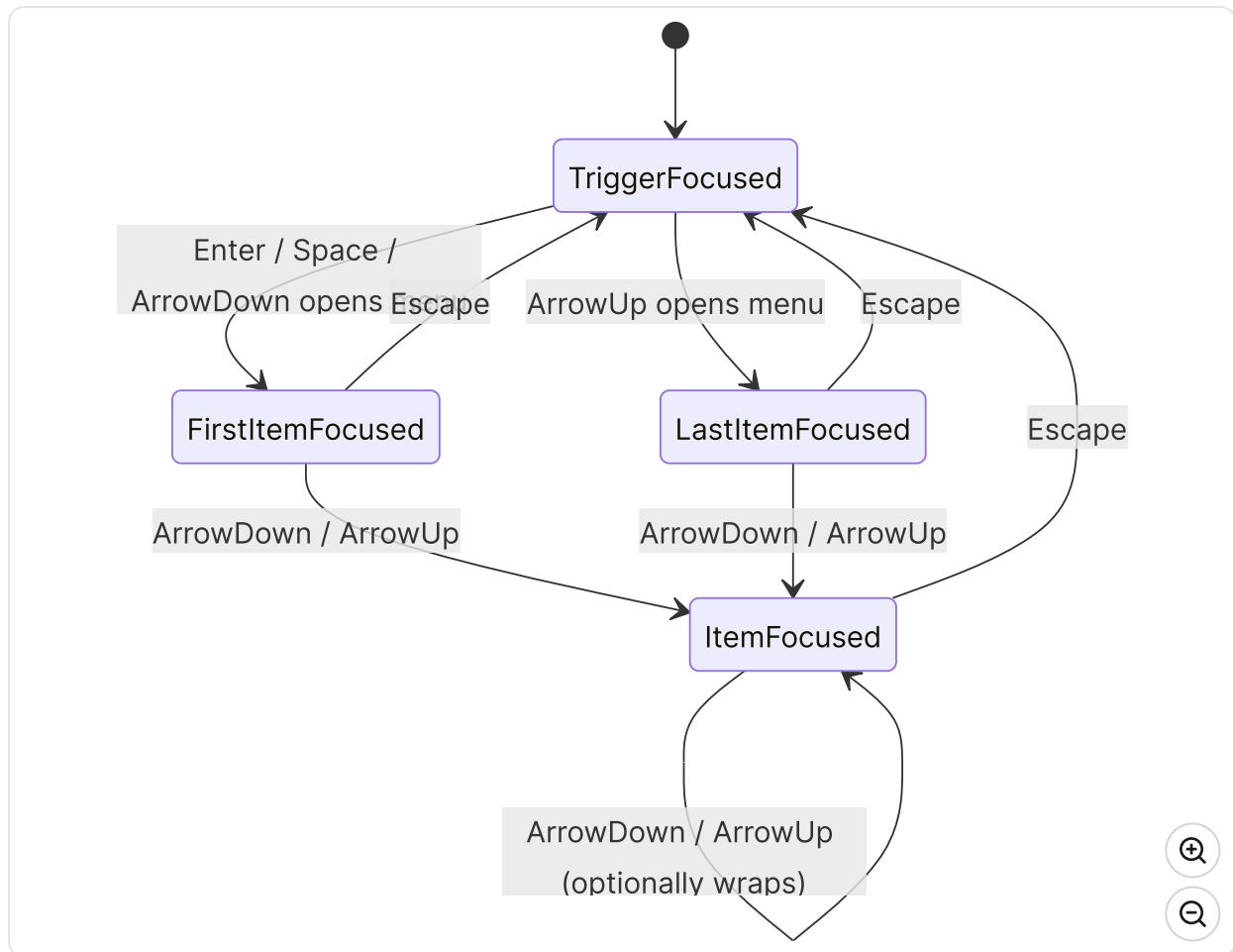
Attach the coordinator's listener only while at least one menu is open and remove it when the group has no open menus.

Here's a simplified React example of click-outside dismissal. The group-aware coordinator above is the implementation contract when simultaneous-open menus are supported.

Focus management

When the menu is open, arrow keys manage focus among menu items. `Tab` and `Shift+Tab` close the menu and continue the normal page focus order; `Escape` closes it

and restores focus to the trigger. An outside click leaves focus on the clicked destination, while item activation may restore the trigger or move focus to the action's resulting context.



Focus transitions across the trigger and menu items

! Restore trigger focus when dismissal calls for it

Escape should return focus to the trigger. Tab, outside clicks, and actions that establish a new destination must be allowed to keep their intended focus movement.

Keyboard interactions

There are two WAI-ARIA patterns to follow: the Menu Button pattern and the Menu pattern. The latter is needed after the menu is open.

Button

When the focus is on the button:

Key	Description
Enter	Opens the menu and places focus on the first menu item.
Space	Opens the menu and places focus on the first menu item.
ArrowDown (Optional)	Opens the menu and moves focus to the first menu item.
ArrowUp (Optional)	Opens the menu and moves focus to the last menu item.

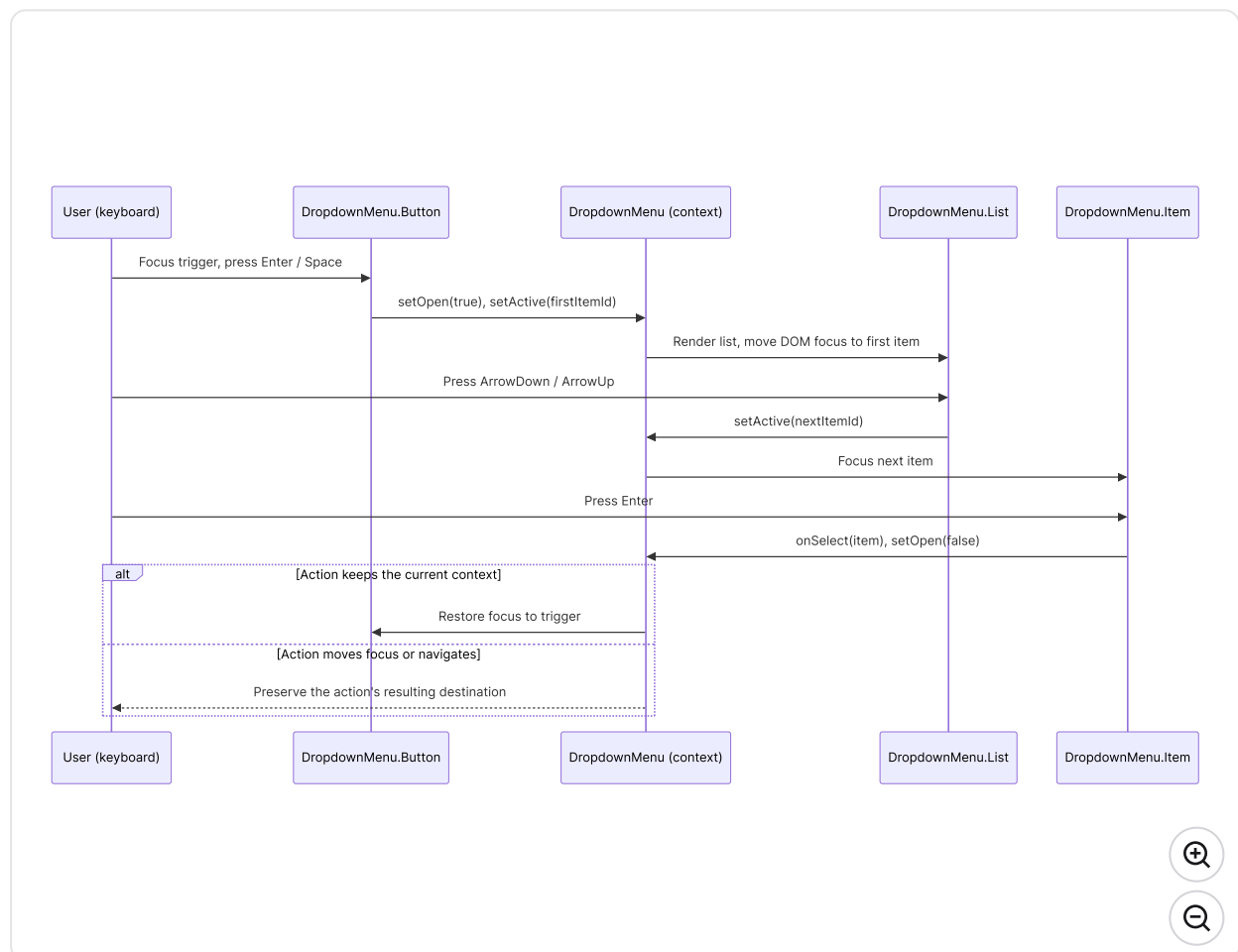
Menu

When the focus is on the menu items:

Key	Description
Enter	Activates the item and closes the menu.
Space (Optional)	Activates the item and closes the menu.
ArrowDown	Moves focus to the next item, optionally wrapping from the last to the first.
ArrowUp	Moves focus to the previous item, optionally wrapping from the first to the last.
Home	If arrow key wrapping is not supported, moves focus to the first item.
End	If arrow key wrapping is not supported, moves focus to the last item.

Key	Description
Esc	Closes the menu that contains focus and returns focus to the button.
Printable characters	Move focus by typeahead using the ordered items' text values.
Tab / Shift+Tab	Close the menu and continue normal page focus order.

End-to-end, a keyboard-driven open, navigate, and activate interaction ties these two patterns together:



Keyboard-driven open, navigate, and select flow

WAI-ARIA roles, states, and properties

The chosen programmatic-focus strategy connects the trigger, menu, and focusable menu items through stable IDs and refs.

- Button
 - The element that opens the menu has role `button`.
 - The element with role `button` has `aria-haspopup` set to either `menu` or `true`.
 - The trigger's `aria-expanded` is `true` while the menu is displayed and `false` while it is closed.
 - The trigger's `aria-controls` refers to the stable ID of the element with role `menu` while it is rendered.
- Menu/Menuitems
 - The element serving as the menu has a role of `menu`.
 - The menu's `aria-labelledby` refers to the stable trigger ID, or the menu has its own `aria-label` when no visible trigger label names it.
 - The items contained in a `menu` have the `menuitem` role, stable IDs, DOM refs, and `tabindex="-1"` so the root can move focus programmatically.
 - When a menu item is disabled, `aria-disabled` is set to `true`.

★ **A styled `<div>` dropdown without `role="menu"` and `menuitem` is not accessible**

Screen readers rely on the button + `aria-haspopup` + `aria-expanded` trio on the trigger, and `role="menu"` with child `role="menuitem"` elements on the list, to announce the component as an actionable menu. Skipping these roles produces a visually working dropdown that is effectively invisible to assistive tech, which is the single most common accessibility failure for this component.

Exercise 1 of 4 Beta

Two dropdowns may remain open, and each menu is portaled away from its trigger. How should a shared coordinator classify a completed click?

- 1 Close all menus as soon as a document-level pointer-down event begins outside the currently active menu.
- 2 Treat every other trigger as outside so clicking it first closes existing menus and then opens the selected sibling.

3

Check the click's composed path for registered triggers and portaled menus; otherwise treat it as outside.

4

Call `root.contains(event.target)` for each dropdown and close every root that does not contain the target.



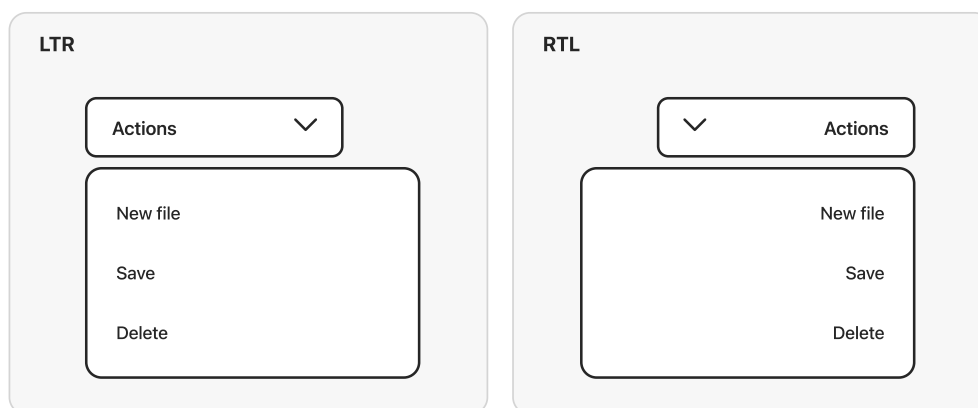
Reveal answer

Check answer ↩

Internationalization (i18n)

Since all user-facing strings are provided by the user, strings can be displayed as-is. However, do note that some strings can be long in certain languages, so overflowing text should either be truncated or wrapped. Text shouldn't overflow outside the menu/button.

For RTL languages, inherit or expose the semantic `dir` value and use logical CSS properties and start/end placement names. Text direction does not imply mirroring every icon or every keyboard command.



Exercise Beta

A dropdown has translated labels that are longer than English and can run in RTL. Which changes preserve its behavior? Select all that apply.

1

Let the popup size or wrap safely for longer translations.

☐

2

Use logical start and end alignment so placement can mirror in RTL layouts.

☐

3

Mirror every icon and reverse the meaning of all keyboard commands whenever `dir="rtl"`.

☐

4

Inherit or expose the menu's semantic `dir` value and use logical CSS properties.

☐

5

Mirror item text in RTL but continue anchoring the popup with physical `left` coordinates.

☐

Reveal answer

Check answer ↩

Summary

A production dropdown is defined by focus restoration, portal rendering, and **WAI-ARIA Menu Button semantics**. Each of those is invisible when the menu is open on a desktop mouse, and each of them breaks the component for keyboard and assistive-tech users the moment the dropdown sits inside a scrollable card or a modal.

Use a compound API backed by shared menu context. `DropdownMenu`, `DropdownMenu.Button`, `DropdownMenu.List`, and `DropdownMenu.Item` keep the trigger, floating surface, and items as distinct subcomponents, while `MenuContext` carries `isOpen`, `activeItemId`, stable trigger/list refs, and the ordered item registry.

Render the list through a portal and make positioning viewport-aware. The list mounts into `<body>` to escape `overflow: hidden` clipping and `z-index` wars, autoflips near viewport edges, and falls back to a scrollable surface via `maxHeight` when the options cannot fit.

Own focus and ARIA behavior explicitly. Arrow keys manage focus inside the menu, Escape restores the trigger, and Tab or an outside click keeps its normal focus destination. The trigger exposes `role="button"`, `aria-haspopup`, and `aria-expanded`, and the list carries `role="menu"` with `role="menuitem"` children so screen readers announce the menu as a menu.

Start with the compound API plus `MenuContext`, then explain the portal, focus restoration, and ARIA Menu Button roles as the decisions that make the component reliable in production.

Exact collision observers, nested-menu coordination, RTL placement edge cases, and a browser/screen-reader test matrix are potential follow-ups rather than part of the core interview implementation.

References

The references below group dropdown menu sources by theme, so it is easier to map each source back to the part of the article it informed.

W3C ARIA authoring practices

This subsection anchors the dropdown design in the authoritative ARIA guidance for menu buttons and menus.

- [Menu button pattern | W3C ARIA](#) for the trigger-side roles, states, and keyboard interactions for a menu button.
- [Menu pattern | W3C ARIA](#) for the menu-side roles, focus order, and keyboard model.

Themed implementations

These libraries ship opinionated visual defaults and are useful when comparing component API ergonomics.

- [Dropdowns | Bootstrap v5.3](#) for a widely used CSS-first dropdown implementation and its markup conventions.
- [React Menu component | Material UI](#) for a themed React menu with full keyboard and accessibility behavior.

Headless implementations

These libraries focus on behavior and accessibility without shipping styles, which is closer to what this article designs.

- [Menu | Base UI](#) for a composable menu with trigger, popup, and item parts.
- [Menu | Ariakit](#) for an unstyled menu with explicit state and composition APIs.
- [Menu | React Aria](#) for customizable menu components and keyboard interaction.

Positioning and floating elements

This subsection covers the browser APIs and libraries used to position the menu popup relative to its trigger.

- [Popover API | MDN](#) for the native browser primitive that handles top-layer rendering and light dismiss.
- [Floating UI](#) for the flip, shift, and collision-aware positioning library commonly used for menus.
- [How to build a dropdown menu with JavaScript | freeCodeCamp](#) for a from-scratch tutorial that covers wiring up toggle, outside click, and keyboard behavior.

Related system design articles

The articles below overlap with dropdown menus on popup positioning, focus management, and dismissal behavior.

- [Modal dialog](#) for overlapping concerns around focus management, portals, and dismissal semantics.
- [Autocomplete](#) for how a menu-like popup pairs with an input and the combobox pattern.

Exercises

Exercise 1 of 16 Beta

A team is defining the first version of a reusable dropdown menu. Which requirements belong in it? Select all that apply.

1

Treat roughly twenty items as a usability preference rather than a hard component limit.

☐

2

Treat text fields, checkboxes, and other nested controls as standard menu-item children in the first version.

☐

3

Support mobile, tablet, and desktop use.

☐

4

Support text and icons as item content, without nested interactive controls inside items.

☐

5

Allow more than one dropdown menu to remain open at the same time.

☐

6

Cap each menu at twenty items and require consumers to move remaining actions into a second menu.

☐

Reveal answer

Check answer ↩

Discussions

↶ ↷ | Normal T_T | B I U <> Aa

Type comment here...

0/1000

Post

Report an issue



✓ Mark complete



Subscribe to our newsletter

Enter your email

Notify me

Practice

Get started

JavaScript functions

User interface coding

System design

Quiz

Guides

Front End Interview Playbook

Front End System Design Playbook

React Interview Playbook

Behavioral Interview Playbook

Study Plans

1 Week Plan

1 Month Plan

3 Months Plan

Company

Pricing

Promotions

Roadmap

About

Team

Contact us

Become an affiliate

Careers

Blog

Medium

DEV Community



English (US) ▾